

Dossiro

A multi-tenant document management system for organisations that must prove what happened to a record, not merely store it. This document describes what Dossiro is, what it does today, what it does not do yet, and what each remaining piece depends on.

STATUS Working build, pre-release	VERIFIED BY 93 assertions	SCHEMA 49 tables, 24 enums
---	---	--

DEFINITION

What Dossiro is

A web application where an organisation files its records, controls who may see each one, shares individual documents outside the company under conditions it sets, and can afterwards prove exactly who did what.

Every customer organisation is a tenant: its own users, its own filing structure, its own records, reachable at its own web address. One deployment serves many tenants and nothing crosses between them.

The distinguishing claim is not storage, which is commodity. It is that the record of activity is evidence. Every consequential action is written to an append-only, hash-chained trail that the database itself refuses to alter, and the chain can be recomputed on demand to prove it has not been touched. An organisation that must answer to an auditor, a regulator or a court can show its working.

ORIGIN

Why it exists

Calm Global evaluated a commercial document management product as a reseller. The evaluation established two things: there is real demand among Nigerian enterprises and public bodies for compliant document management, and the available products are desktop-era software sold at licence costs that assume a different market.

Dossiro is the response. A browser-first system built for organisations that need the compliance posture without the desktop deployment, priced and hosted for the market it serves. It is designed to be sold as a managed service, with self-hosting available for buyers whose regulator requires it.

USERS

Who uses it

Three populations touch the system, and the boundaries between them are the thing most worth getting right. Roughly half of the verification suite consists of proving what each population cannot do.

POPULATION	WHO THEY ARE	WHAT THEY CAN REACH
Platform operator	Calm Global staff running the service	Creates, suspends and closes tenants. Cannot read any tenant's documents; the platform organisation is barred by the database from holding a document at all.
Tenant user	Staff at a customer organisation	Everything their role and grants allow, inside their own tenant only.
External recipient	A counterparty holding a share link. No account.	Exactly one document, under the conditions set on that link, and nothing else.

Tiers within a tenant

A tier is the ceiling on what a person may ever do. Grants on individual folders and documents then decide what they actually reach.

TIER	INTENDED FOR	NOTABLE LIMIT
SYSTEM_ADMIN	Technical owner	Full control, including permanent purge and audit export
ORG_ADMIN	The organisation's administrator	Everything except purging a record under legal hold
MANAGER	Head of a department	Approves, grants access, shares confidential records externally
CONTRIBUTOR	Staff who file day to day	Creates and edits; cannot grant access to others
VIEWER	Read-only staff	Reads what they are granted; files nothing
EXTERNAL	Contractors and partners with accounts	Only what is explicitly granted, nothing by inheritance

STRUCTURE

The three realms

Operating the platform and using the product are different jobs, so they are different places.

1. The operator console

Where Calm Global administers the business: provisioning tenants, suspending them for non-payment, issuing licences. Suspending or closing a tenant locks a whole company out of its own records, so it requires a written reason, and that reason is recorded in both the platform's audit trail and the tenant's own, where nobody can later edit it.

2. The tenant workbench

Where the customer's staff work: the filing cabinet, search, sharing, and the audit trail. This is the product as most people experience it.

3. Tenant administration

Where a customer administers themselves: people, roles, departments, branches, web addresses. Currently a section of the workbench; splitting it into its own area is an agreed next step.

A DELIBERATE SEPARATION

Calm Global appears twice: once as the operator, and once as an ordinary tenant holding its own records. Those are separate organisations with separate data. The operator console cannot read the records tenancy, and vice versa.

SCOPE

The workbench, area by area

The workbench has eleven areas. Five run on real data. The rest are designed and modelled but not yet connected, and the interface says so on every screen rather than showing convincing sample data unlabelled.

WHY THE LABELLING MATTERS

Sample data is a demo hazard: someone opens Invoices, sees six invoices, and believes them. Every area that is not live is marked in the interface, and panes that still show the designer's illustration beside a genuine record are marked individually.

AREA	STATE	WHAT IT DOES
Repository	LIVE	The filing cabinet. Folders nest without limit; documents carry immutable version history, classification and an owner.
Search	LIVE	Full-text search inside document bodies, ranked, with the matched phrase quoted. Scanned pages are not searchable until OCR exists.
Sharing	LIVE	Every share link in the organisation, filtered by state: active, expired, download cap reached, revoked.

AREA	STATE	WHAT IT DOES
Audit	LIVE	The event log, filtered in the database, with the hash chain verified alongside every page.
Administration	PART LIVE	People, roles, branches and web addresses are live. Remaining scopes are still sample.
Capture	NOT BUILT	Scanning and device intake. Needs the OCR pipeline.
Ingest	NOT BUILT	Channel intake. WhatsApp and email intake work at the API level but this screen is not connected to them.
Invoices	NOT BUILT	Invoice extraction and matching. The data model is complete; no extraction runs.
E-forms	NOT BUILT	Form builder and submissions. Schema only.
HR	NOT BUILT	HR document handling. Blocked behind user-defined document types.
Approvals	NOT BUILT	Workflow routing. Workflow tables exist; nothing drives them.

Verified working today

- Folders with unlimited nesting, move, and subtree delete
- Documents with immutable version history and restore
- Role and classification based access control, with inheritance and explicit deny
- Share links with expiry, download caps, access codes and view-only enforcement
- Metadata and full-text search with date-range filters
- Hash-chained audit trail with integrity verification
- Per-page reading analytics on shared documents
- Recycle bin, restore, and legal hold
- WhatsApp and email upload and download with verified sender identity
- A delta sync feed for offline clients (reading side)

CORE MODEL

How access is decided

Two independent systems both have to say yes: permissions, which come from a person's role and say what kind of thing they may do at all, and grants, which are attached to folders and documents and say which specific records they may touch.

Levels

A grant carries one level, and the levels are ordered: NONE, READ, DOWNLOAD, WRITE, APPROVE, MANAGE, OWNER. DOWNLOAD is deliberately separate from READ: being allowed to look at a contract on screen is not the same as being allowed to take a copy away.

Resolution

A grant can be made to a person, a department, or a role, and can be attached to a document or to a folder, where it flows down to everything beneath. When several apply, the most specific scope wins: a rule on the document beats one on its folder, which beats one on the folder above. At equal specificity, deny beats allow.

THE CASE THAT PROVES IT

A clerk in Accounts Payable inherits write access to the whole Finance cabinet through their department, but carries an explicit deny on the Invoices subfolder. They can file into Finance and are refused on Invoices. This is asserted on every verification run.

Classification

Independently of who you are, every document carries a classification (PUBLIC, INTERNAL, CONFIDENTIAL or RESTRICTED) and the classification constrains what may be done with it regardless of permission. A RESTRICTED record cannot be shared outside the organisation at all; a CONFIDENTIAL one cannot be shared as an open link with no named recipient, and only a manager or administrator may share it externally.

A folder that is marked as not inheriting access is sealed: grants from above do not reach into it, and only explicit grants apply. This is how an HR cabinet stays closed to an administrator who can otherwise see everything.

THE DIFFERENTIATOR

Evidence and the audit trail

Every consequential action writes an entry. Each entry hashes its own content together with the hash of the entry before it, so the log forms a chain: change or remove any entry and every subsequent hash stops agreeing. An integrity endpoint recomputes the chain and reports the first entry that disagrees.

The guarantee does not rest on application code being well behaved. A database trigger rejects UPDATE and DELETE on the audit table outright, so no role, including the owner, and including anyone with direct database access through the application's own credentials, can quietly edit history. Both refusals are asserted on every verification run.

Writes are serialised per organisation with an advisory lock, so two simultaneous actions cannot chain off the same predecessor and fork the history.

KNOWN LIMIT, STATED PLAINLY

The hashed content covers the action, the actor and the resource. It does not yet cover the originating address, the resource name, or the metadata payload. Those fields cannot be edited in practice because the trigger blocks it, but they are not cryptographically bound. Widening the hash would invalidate every existing entry, so this is a decision to take before real customer data lands.

EXTERNAL ACCESS

Sharing outside the organisation

A share link lets someone without an account reach exactly one document. The person issuing it sets the terms: an expiry, an optional access code, a download cap, whether downloading is permitted at all, whether the document is watermarked, and which email addresses may open it.

Opening a protected link is two steps. The recipient first authorises with the access code and receives a short-lived signed ticket; the bytes are only served against that ticket. A ticket issued for one link does not work on another. View-only is enforced at the point the bytes are served, not merely hidden in the interface: requesting the same document as an attachment is refused.

Revocation is immediate. A revoked link returns gone on the next request, and every open is written to the audit trail.

HOW IT IS BUILT

Architecture

PART	BUILT WITH	NOTES
API	NestJS 11 · TypeScript 5.7	18 modules, port 4010
Web app	React 18 · Vite 6	The workbench, port 3015
Marketing site	Next.js 14	Separate deployment, port 3020
Database	PostgreSQL 18	49 tables, 24 enum types
Database access	node-postgres 8.23	No ORM. Hand-written parameterised SQL.
Storage	Local filesystem	Driver is pluggable; S3 is a decision not yet taken

No ORM, deliberately

Every query is written by hand as parameterised SQL. The access-control resolution alone involves recursive folder walks, department hierarchies and specificity ordering; expressing that through an ORM's abstraction obscured what the database was actually being asked to do. Row types are generated from the live schema, so a column that changes shape breaks the build rather than failing at runtime.

Three details worth knowing

- Transactions propagate implicitly. A nested transaction joins the one already running rather than deadlocking against it, so a service can be called standalone or inside a larger operation without knowing which.
- Timestamps are UTC end to end. The Postgres driver reads and writes timestamps in the machine's local time by default, which silently shifts every date by the server's offset. This is corrected in three places at once: the type parser, parameter serialisation, and the session timezone.
- Request origin is ambient. The address and user agent are carried in request-scoped context, so an audit entry records where an action came from without each call site remembering to pass it.

Verification

93 assertions across five suites, run against a live database and a running API rather than mocks. They exist to prove the guarantees the product is sold on, not to chase coverage. Notably they include what the database enforces on its own: the append-only trigger, foreign keys, and the platform organisation's inability to hold a document.

SCHEMA

The data model

49 tables. The ones that matter most to understanding the system:

TABLE	HOLDS
organizations	Tenants, plus the single platform organisation that runs the service

TABLE	HOLDS
users	People. Email is unique per organisation, not globally, so a consultant can belong to two tenants.
folders	The cabinet tree, with a materialised path for subtree queries
documents	Records, each pointing at a current version
document_versions	Immutable history. A new edition never overwrites the previous one.
document_index	Extracted text for full-text search, separate from the document row
access_grants	Who may reach what, at which level, allow or deny
share_links	External links and their conditions
audit_events	The hash chain. Append-only, enforced by trigger.
tenant_hostnames	Which web address belongs to which tenant

Identifiers are generated in the application rather than by the database, so a row's id is known before it is written, which is what lets a folder's materialised path be correct in the insert itself rather than needing a follow-up update.

Migrations are plain SQL files applied by a small runner that records a checksum of each. A migration edited after it has been applied is detected and refused, rather than silently diverging between environments.

Licensing

Licences are Ed25519-signed and verified offline, so a self-hosted deployment with no outbound connectivity still enforces its terms. Seat entitlement is read from the signature, never from a column in the database: editing the seat limit directly changes nothing, which is asserted on every verification run.

ROADMAP

Not built yet

Each of these is modelled in the database and has an agreed approach. What is listed is what must be true before it can be built, so that the decision is not re-litigated later.

OCR and batch PDF conversion

SCHEMA READY

The gateway feature. Several other items are blocked behind it, because they need text that only OCR can produce.

NEEDS

Redis for the job queue · a Python OCR service · a conversion container

APPROACH

Job rows drive a queue. OCR runs in a separate Python service; conversion runs headless. Results feed the search index.

Intelligent indexing, naming and summarisation

SCHEMA READY

NEEDS

An AI provider key · the pgvector extension · OCR text available

APPROACH

Extracted text is chunked and embedded for semantic search. A model proposes a name, folder and tags for human confirmation; it never renames files unattended.

Invoice and HR document extraction

SCHEMA READY

NEEDS

The OCR pipeline · a human review queue

APPROACH

Line items get real columns so they can be aggregated and reconciled. Every extraction stays unverified until a person confirms it.

Workflow routing and approvals

SCHEMA READY

NEEDS

Notification delivery · a workflow builder interface

APPROACH

A definition is an ordered list of steps; each step creates a task for a person, department or role, and confers temporary access for the duration of that task.

E-signatures

SCHEMA READY

NEEDS

PDF flattening · a certificate store for the later phase

APPROACH

Signature positions are stored as fractions of the page so they survive any zoom. Drawn signatures first; certificate-based signing later. Every signature carries an evidence bundle for disputes.

E-forms

SCHEMA READY

NEEDS

A form builder interface

APPROACH

A form definition drives a renderer; a submission can merge into a template and start a workflow.

Live co-editing and Microsoft/Adobe integration

BUY, DON'T BUILD

NEEDS

An embedded document server, or Microsoft registration

APPROACH

Real-time co-authoring is embedded from an existing product. Building a collaborative editor is out of scope at any realistic budget, and this decision is recorded so it is not revisited.

Full offline editing

SCHEMA READY

NEEDS

A desktop or mobile client · a conflict resolution model

APPROACH

Offline reading and queued uploads already work through the sync feed. Bidirectional offline editing needs a version-vector model and its own client; it is a separate phase, not an extension of what exists.

SOC 2 readiness

MOSTLY NOT CODE

NEEDS

An auditor engagement · vendor agreements · a penetration test · 3 to 12 months of evidence

APPROACH

The technical control surface is largely in place: hash-chained audit, encryption in transit, classification, retention, legal hold, session revocation. Certification itself is a programme, not a feature. Most buyers asking for SOC want SOC 2 Type II, which covers security; SOC 1 covers financial reporting controls.

The largest gap

USER-DEFINED DOCUMENT TYPES

Customers expect to define their own kinds of document, a Contract or a Personnel File, each with its own typed, indexed fields they can search on. The schema does not express this yet, and the HR area is blocked behind it. It is the clearest functional gap against established products in this market.

DELIVERY

Deployment

Two models, the same software.

Managed. The default. A customer gets their own tenancy on Calm Global's platform at their own web address. Nothing to install, nothing to maintain.

Self-hosted. For buyers whose regulator requires records to stay on their own infrastructure. Licences verify offline precisely so this works without outbound connectivity.

A tenant's web address is what identifies them at sign-in. Visiting a tenant's own address settles which organisation is being signed into before anyone types anything; the shared sign-in asks only when the address settles nothing and the person belongs to more than one organisation.

PRACTICAL

Running it locally

```
npm run db:migrate      # apply migrations
npm run db:seed         # 278 documents across 17 folders
npm run db:reset        # drop, rebuild, reseed (guarded to local only)
npm run verify          # 93 assertions
```

The workbench runs at localhost:3015, the API at localhost:4010. Seeded tenants also answer on their own addresses (acme.localhost:3015, harbor.localhost:3015, records.localhost:3015), which is how the hostname-identifies-the-tenant behaviour can be exercised locally.

Seeded accounts

EMAIL	PASSWORD	ORGANISATION AND ROLE
admin@acme.test	Dossirol2026	Acme Corporation — administrator, sees everything
manager@acme.test	Dossirol2026	Acme — manager, Finance and Shared
clerk@acme.test	Dossirol2026	Acme — contributor, denied on Invoices 2026
client@partner.test	Dossirol2026	Acme — external, Shared only, read-only
ada@harborfreight.test	HarborFreight!2026	Harbor Freight — a second tenant, for isolation

Development credentials for a local database only. They are in the seed file and are not secrets.

OUTSTANDING

Open decisions

Storage backend

Whether to add S3 as a storage driver. This is genuinely two separate questions that are easy to conflate: using object storage as the backend for document bytes, and exporting documents to a customer's own bucket as a published deliverable. The second must be classification-aware, because publishing a restricted

record to a bucket the organisation does not control would defeat the entire access model.

The audit hash canonical

Whether to widen the hashed content to cover the originating address and metadata. Doing so invalidates every existing entry, so it should be settled before real data lands.

Tenant administration split

Agreed in principle: move tenant self-administration out of the workbench into its own area. Not started.

Preview and editing

The document preview and editing panes are still the designer's illustration. A real preview needs the content stream and a viewer; editing is the buy-not-build decision recorded above.

Dossiro · Calm Global · Lagos and Abuja. This document describes a working pre-release build as of 19 September 2026. Where something is not built, it says so.